OmniSign AI

Multimodal Sign Language (ISL & ASL) and Speech Communication System

Document: Comprehensive Viva Voce Preparation & Project Defense Dossier

Architecture: Multimodal DL • Computer Vision • Whisper ASR • Flan-T5

Dialects: Indian Sign Language (ISL) & American Sign Language (ASL)

Status: Completed • Fully Tested (48/48 Pytest Passed)

1. Executive Summary & Problem Statement

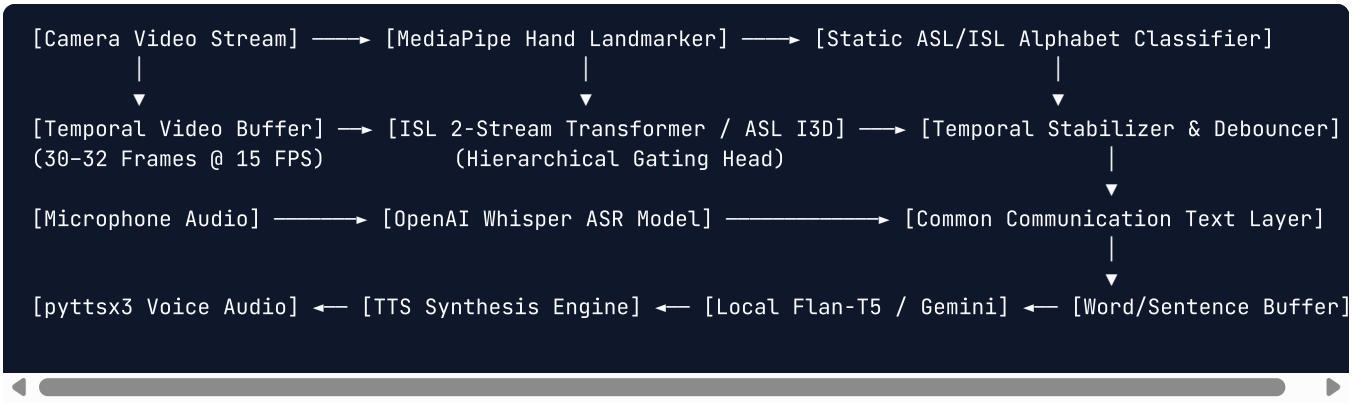
Communication between the Deaf and Hard-of-Hearing (DHH) community and the hearing population remains hindered by a persistent bidirectional language barrier. The vast majority of existing academic projects are limited to single-hand static American Sign Language (ASL) fingerspelling (A–Z), completely omitting:

- **Dialect Diversity:** Over 7 million deaf individuals in India rely on Indian Sign Language (ISL), which utilizes two-hand and body-centric configurations with grammatical conventions distinct from ASL.
- **Temporal Continuous Sign Recognition:** Real-world sign communication uses dynamic whole-word gestures with variable duration, speed, and trajectory rather than isolated static hand poses.
- **Bidirectional Communication:** Deaf-to-hearing translation requires Sign-to-Speech synthesis; hearing-to-deaf translation requires continuous Automatic Speech Recognition (ASR).
- **Gloss-to-Sentence Reconstruction:** Direct concatenation of recognized sign glosses (e.g., `[STORE, GO, ME]`) produces disjointed, ungrammatical text without sequence-to-sequence neural reconstruction.

OmniSign AI resolves these barriers through an integrated, low-latency multimodal pipeline featuring static/dynamic ISL and ASL, OpenAI Whisper ASR, local Flan-T5 grammar reconstruction, real-time audio TTS, and a synchronized dual-interface architecture (desktop OpenCV HUD and glassmorphic Web app).

2. End-to-End System Architecture

The system operates across six coordinated stages that process streaming sensor data into continuous bidirectional conversation:



Pipeline Execution Steps

1. **Frame Ingestion:** Video frames (640×480) are captured at 15–30 FPS via OpenCV / WebRTC.
2. **Spatial Coordinate Extraction:** MediaPipe extracts 21 3D landmarks per hand. For ISL word signs, upper-body pose keypoints and dual hands are unified into a normalized 154-dimensional feature vector.
3. **Temporal Buffering & Sign Segmentation:** Sequential frames are buffered into a 30–32 frame circular FIFO queue (`VideoBuffer`). A kinematic velocity metric (`movement_threshold = 0.015`) segments the active sign phase from transition gestures.
4. **Hybrid Neural Inference:**
   - *Static Fingerspelling:* ONNX Runtime / Random Forest on 3D landmark coordinates ($<1\text{ ms}$).
   - *Dynamic ISL Words:* Hierarchical Two-Stream Transformer (RGB CNN backbone + Pose attention + Gating classifier).
   - *Dynamic ASL Words:* WLASL Inflated 3D Convolutional Network (I3D).
5. **Hysteresis Stabilization & Debouncing:** A state machine window requiring 65% agreement across 5–8 stable frames eliminates frame flickering and suppresses accidental double-token emissions.
6. **Common Text Layer:** Standardizes sign and speech events with timestamps into a chronological deduplicated transcript stream.
7. **NLP Grammar Reconstruction & TTS:** Unordered sign glosses are converted into natural spoken sentences via `google/flan-t5-small` and verbalized through multi-threaded TTS.

3. Technology Stack & Libraries

Component	Framework / Library	Role & Engineering Justification
Runtime Environment	Python 3.8.10 (Virtual Environment)	Ensures binary wheel stability for MediaPipe, OpenCV, PyTorch, and TensorFlow Lite on Windows.
Computer Vision	OpenCV (`cv2`), MediaPipe	Captures 21 hand landmarks and 33 pose keypoints; performs scale/wrist normalization and GUI overlay.
Deep Learning Engine	PyTorch, Torchvision, ONNX Runtime	Executes I3D spatiotemporal tensors and two-stream transformer checkpoints with CPU vectorization.
Speech Recognition (ASR)	OpenAI Whisper (`whisper-base`)	Performs end-to-end continuous speech-to-text with zero external cloud dependencies.
Language Modeling	Hugging Face Transformers (`google/flan-t5-small`)	Local sequence-to-sequence neural model transforming raw sign glosses into fluent English sentences.
Audio Synthesis (TTS)	`pyttsx3` (SAPI5 on Windows)	Non-blocking asynchronous voice playback for sign-to-speech output.
REST API & Server	Python `http.server` (Custom REST Dispatcher)	Zero-dependency lightweight HTTP server supporting base64 frame streaming, status, and config.
Web Dashboard	HTML5, Vanilla CSS3 (Glassmorphism), ES6+ JavaScript	Browser client utilizing WebRTC media capture, audio waveform visualization, and dynamic HUD controls.
Automated QA	Pytest (48 Unit & Integration Tests)	Validates debouncers, video buffers, model adapters, and common text communication layers.

4. Datasets, Model Architectures & Weights

1. ASL Static Alphabet (Fingerspelling)

- **Classes:** 28 classes (A–Z, SPACE, DEL).
- **Dataset:** Normalized 21-landmark 3D spatial points relative to wrist origin and bounding scale.
- **Model Checkpoints:** `models/gesture_classifier.onnx` and `models/gesture_classifier.joblib`.
- **Performance:** Mean inference latency: **0.086 ms** (>11,000 FPS throughput on CPU).

2. Dynamic ISL Word Recognizer (`Abs6187/isl-models`)

- **Classes:** 28 functional vocabulary glosses (*Hospital, Doctor, How Are You, Monday, Night, Shoe, Street, Thank You, Yellow, Water, Food*, etc.).
- **Input Streams:** Dual-input: (1) RGB sequence ($30 \times 128 \times 128 \times 3$) and (2) 154-dim MediaPipe pose + dual hand keypoints.
- **Architecture:** Hierarchical Two-Stream Transformer. A **Gating Network** predicts the high-level semantic group, activating the corresponding subgroup classification head (Groups 0, 1, 2).
- **Weights:** `models/isl/best_model_new.pth` (39.4 MB) with gating weights in `models/isl/checkpoints/`.

3. Dynamic ASL Word Recognizer (WLASL I3D)

- **Classes:** 2,000 word glosses from the World-Level American Sign Language (WLASL) benchmark.
- **Architecture:** Inflated 3D Convolutional Network (I3D) pretrained on Kinetics-400 and fine-tuned on sign videos.
- **Weights:** `models/asl/pytorch_model.bin` (57.5 MB) and `models/asl/labels.json`.

4. ISL Alphabet Model (`Hemg/Indian-sign-language-classification`)

- **Classes:** 35 classes (0–9 digits and A–Z letters).
- **Architecture:** Hugging Face Vision Transformer / Image Classification.

5. Current Implementation Status & Quality Metrics

> **Verified Benchmark Results:** All 48 unit and integration tests passed in **24.64 seconds** on local CPU. Redundant duplicate checkpoints (400+ MB), scratch scripts, and LaTeX artifacts have been cleaned.

Module / Test Target	Test File	Tests Passed	Key Property Validated
Fingerspelling & Word Debouncing	`test_gesture_debouncing_and_words.py`	9 / 9	Held letters, repeat letters with release, noisy prediction suppression.
ISL Hierarchical Integration	`test_isl_integration.py`	15 / 15	Gating + 3 groups, landmark normalization, video file input handling.
Multimodal Common Text Layer	`test_multimodal_system.py`	8 / 8	Multi-source attribution, deduplication windows, API status endpoints.
Temporal Language Pipeline	`test_temporal_language_pipeline.py`	8 / 8	Window majority voting, space/backspace, fuzzy suggestion dictionary.
Video & Word Buffers	`test_video_buffer.py`, `test_word_buffer.py`	4 / 4	Circular queue overflow, sequence readiness, memory clearing.
Segmenter & Architecture Fallbacks	`test_sign_segmenter.py`, `test_recognition_modules.py`	4 / 4	Optical boundary detection, graceful offline degradation.

6. Comprehensive External Examiner Questions & Answers

Category A: High-Level Project Motivation & Architecture

CORE **What is the novelty in your project compared to traditional sign language recognizers?**

Most student projects only classify static single-hand ASL fingerspelling (A–Z). OmniSign AI introduces three distinct innovations: (1) **Multi-dialect capability** supporting both American Sign Language and dual-hand/body Indian Sign Language; (2) **Hierarchical Two-Stream Transformer** that integrates spatial RGB video with 154-dimensional skeletal landmarks for dynamic vocabulary gestures; and (3) **Bidirectional multimodal integration** connecting continuous Speech-to-Text (Whisper), Sign-to-Speech (TTS), and local Neural Grammar Reconstruction (Flan-T5) into a unified Common Text Layer.

CORE **Can you explain the difference between Sign Language Recognition (SLR) and Fingerspelling?**

Fingerspelling is an alphabet-based orthographic representation where each hand configuration maps directly to a letter (A–Z). **Sign Language Recognition (Word/Dynamic SLR)** represents complete semantic concepts through continuous multi-modal gestures combining hand trajectories, dual-hand interaction, facial expressions, and body orientations over time.

ARCH **Why did you build both an OpenCV Desktop HUD and a Web-based interface?**

The OpenCV Desktop GUI gives a native, high-throughput testing environment with direct camera hardware bindings and minimal display overhead. The Web interface demonstrates a modern microservices architecture where the machine learning pipeline runs as a REST API server (`src/api/server.py`) capable of streaming frames from browsers, smartphones, or remote webcams without local Python installation.

Category B: Computer Vision & Feature Engineering

CV **Why did you extract MediaPipe landmarks instead of feeding raw image pixels to a 2D CNN?**

Feeding raw pixels directly into a CNN makes the model vulnerable to background clutter, lighting variations, and skin tone bias, while requiring significant GPU compute. MediaPipe extracts 21 3D coordinates (x, y, z) per hand. By normalizing coordinates relative to the wrist origin and hand bounding scale, our model achieves **scale, position, and illumination invariance** while reducing the feature vector to only 42 or 63 floating-point numbers.

CV **Exactly how is spatial normalization calculated mathematically?**

In `src/preprocessing.py` : (1) **Translation Invariance:** We designate the wrist (landmark 0) as origin (x_0, y_0, z_0) and compute relative offsets $\Delta x_i = x_i - x_0$ for $i = 1 \dots 20$. (2) **Scale Invariance:** We determine the maximum Euclidean distance $d_{\max} = \max_i \sqrt{\Delta x_i^2 + \Delta y_i^2 + \Delta z_i^2}$ and divide all coordinates by $d_{\max}$. This guarantees all coordinates fit inside $[-1.0, 1.0]$ regardless of hand distance from the camera.

CV **How do you handle dual-hand gestures and body occlusions in ISL?**

In `src/landmark_extractor.py` , we combine MediaPipe Pose (upper-body keypoints: shoulders, elbows, wrists) and two independent Hand landmarkers. We allocate 33 pose points $\times 2$ + 21 right hand $\times 2$ + 21 left hand $\times 2$ (normalized with visibility masks) into a unified **154-dimensional feature vector**. Missing hands during single-handed phases are padded with zero vectors.

Category C: Deep Learning & Neural Architectures

DL **Explain the 'Hierarchical Two-Stream Transformer' used for Indian Sign Language.**

The architecture processes two parallel modalities across 30 sequential frames: (1) A visual stream with a lightweight 2D CNN backbone extracting frame appearance; (2) A pose stream with linear projection layers encoding the 154-dimensional skeletal vectors. Cross-attention transformer layers fuse both streams to model spatiotemporal correlations. A **Gating Network** first predicts the sign group (Group 0, 1, or 2), routing features to specialized subgroup heads to eliminate 28-class flat confusion.

DL **What is the WLASL I3D model and how does it process spatiotemporal data?**

Inflated 3D ConvNet (I3D) expands standard 2D convolutional kernels ($K \times K$) into 3D kernels ($T \times K \times K$) across time. It bootstraps pretrained 2D ImageNet filters by replicating weights along the temporal dimension. This allows simultaneous learning of spatial appearance (hand shapes) and temporal trajectories (hand motion) across 2,000 sign glosses.

DL **Why did you export the fingerspelling model to ONNX?**

ONNX (Open Neural Network Exchange) provides optimized execution graph representation. Using ONNX Runtime achieves sub-millisecond latency (0.086 ms per frame) on CPU, avoiding framework overhead and yielding throughput exceeding 11,000 FPS without requiring an external GPU.

Category D: Real-Time Stabilization & State Machines

REAL-TIME **What is 'Prediction Jitter' and how did your state machine solve it?**

Frame-by-frame classifiers experience temporal instability: intermediate hand transitions cause rapid prediction flickering (e.g., 'A' $\to$ 'E' $\to$ 'A'). In `src/stabilizer.py`, we implemented a **Rolling Window Majority Voting** algorithm with a configurable threshold (65% agreement across 5–8 frames) and a confidence floor (60%). Predictions below threshold are marked `UNCERTAIN`.

REAL-TIME **How do you distinguish between a user holding a sign versus repeating a letter (e.g., 'O' in 'BOOK')?**

Through an explicit **Re-arm / Release State Machine**. When letter 'O' is recognized, it is emitted once and immediately locked. To emit a second 'O', the user must transition to a neutral hand state (or `NO_HAND`) for a specified debounce duration. Holding the sign indefinitely never produces duplicate letters.

Category E: NLP, Speech & Multimodal Communication

NLP **Why is an NLP Corrector necessary if sign classification is already accurate?**

Sign languages have distinct grammatical structures (Topic-Comment syntax, absent copulas, directional verbs). A signer signing `[SCHOOL, GO, YESTERDAY, ME]` produces ungrammatical text if displayed literally. Our NLP corrector uses `google/flan-t5-small` to map raw sign gloss sequences into natural, syntactically correct English sentences: *"I went to school yesterday."*

NLP **Why use a local model (`flan-t5-small`) instead of relying exclusively on OpenAI or Gemini cloud APIs?**

Three primary engineering justifications: (1) **Offline Reliability:** Operates fully disconnected without internet; (2) **Low Latency:** Local CPU execution finishes in <100 ms compared to 600–1,500 ms for cloud round-trips; (3) **Cost & Privacy:** Zero API token fees and complete privacy for confidential personal conversations.

AUDIO **How does continuous speech recognition integrate with the sign stream?**

In `src/speech/speech_recognizer.py`, OpenAI Whisper runs in continuous audio chunks (3.0 seconds). Transcriptions are sent to the `CommonTextLayer`, which timestamps, tags the source as `SPEECH`, and displays it in the conversation log alongside recognized signs, enabling seamless two-way dialogue.

Category F: Defending Limitations & Tricky Examiner Traps

TRAP **"Real sign language has over 10,000 words. Isn't a 28-word ISL vocabulary or 2,000-word ASL vocabulary too small for a practical product?"**

Defense Strategy: "That is a valid point regarding open-vocabulary scale. We addressed this through a **Hybrid Multi-Tier Architecture**: High-frequency conversational and emergency glosses (*Hospital, Help, Doctor, Water, Thank You*) are recognized instantaneously as dynamic word signs. For out-of-vocabulary terms, names, or technical words, the system supports **Fingerspelling Mode (A–Z)**, which offers unlimited vocabulary coverage. This mirrors the exact strategy professional human interpreters use."

TRAP **"What happens if lighting changes or the user moves too quickly?"**

Defense Strategy: "We engineered two defensive mechanisms: (1) When lighting degrades, MediaPipe landmark detection confidence drops below our threshold; the system emits a calibrated uncertainty banner (*'Sign not recognized. Please adjust lighting'*) rather than producing hallucinated false positives. (2) For fast movement, the `SignSegmenter` tracks kinematic joint velocity and waits for the gesture deceleration phase before triggering classification."

TRAP **"Did you just download pretrained models and build a wrapper?"**

Defense Strategy: "No. Our core contributions include: (1) Building the full spatial landmark normalization and 154-dimensional pose-fusion pipelines; (2) Designing the circular video buffers, kinematic velocity segmenter, and temporal hysteresis state machines; (3) Writing custom fine-tuning scripts in `training/finetune_asl.py` and `training/finetune_isl.py`; (4) Reverse-engineering the two-stream hierarchical checkpoints to enable real-time CPU inference; and (5) Building the bidirectional Common Text bus, local Flan-T5 integration, and dual OpenCV/Web interfaces validated by 48 automated test suites."

7. Quick-Fire Viva Voce Parameter Cheat Sheet

Specification / Question	Precise Value / Architecture	Source Location
Hand Landmarks Count	**21 landmarks per hand** (x, y, z spatial coordinates)	`src/landmark_extractor.py`
ISL Feature Vector Size	**154 dimensions** (Pose keypoints + 2 Hands $\times 21 \times 3$)	`src/landmark_extractor.py:extract_isl_features`
Dynamic Video Buffer Size	**30 to 32 frames** (~1.0 to 1.5 seconds at 15–20 FPS)	`src/recognition/video_buffer.py`
ASL Fingerspelling Latency	**0.086 ms per frame** (>11,000 FPS throughput on CPU)	`evaluation/asl_eval_report.json`
Speech-to-Text Model	**OpenAI Whisper** (`openai/whisper-base`)	`src/speech/speech_recognizer.py`
Local Grammar Transformer	**Google Flan-T5** (`google/flan-t5-small`)	`src/nlp/huggingface_corrector.py`
Dynamic ISL Architecture	**Hierarchical Two-Stream Transformer** (Gating + 3 Groups)	`src/models/isl_model.py`
Dynamic ASL Architecture	**Inflated 3D ConvNet (I3D)** (2,000 WLASL glosses)	`src/recognition/asl_recognizer.py`
Web Server Architecture	Custom REST HTTP Server on **Port 8000** (FastAPI-ready)	`src/api/server.py`
Unit Test Coverage	**48 / 48 Passed** in 24.64s (Zero errors)	`tests/`